Wrocław 2025.11.03
Autorzy: Aleksander Stepaniuk (*KN GANGStAS*), Marcin Komarnicki i Michał Przewoźniczek

Dziewiąty Konkurs Programowania Obiektowego
Czyli na co komu C++ i obiekty?
Edycja 2026 – Optymalizacja zestawu tras pojazdów dostawczych

I. Cel i warunki konkursu

Celem konkursu jest dobra zabawa w rozwiązywanie łamigłówek, które jednocześnie są zadaniem otwartym. Zaproponowana metoda do rozwiązywania określonej w punkcie 3 klasy zadań, może używać dowolnych mechanizmów, pod warunkami wskazanymi poniżej.

1. **Wymagania dla optymalizatora.** Optymalizator musi dać sklasyfikować się jako algorytm genetyczny (wymogi 1.1 i 1.2), oraz musi posiadać wszystkie cechy wyszczególnione poniżej.

   1.1. **Populacja rozwiązań.** Optymalizator musi posiadać populację osobników/rozwiązań, kodujących na ciągu dyskretnym (ciąg wartości całkowitych dodatnich, gdzie każda zmienna ma taką samą dziedzinę) o odpowiedniej długości rozwiązanie problemu.
   1.2. **Operatory genetyczne.** Osobniki-rozwiązania muszą wchodzić ze sobą w interakcje, wymieniając ze sobą informacje np. za pośrednictwem krzyżowania lub innych operatorów.
   1.3. **Struktura optymalizatora oparta o iteracje**. Dla całego optymalizatora (algorytmu genetycznego), poza inicjacją, można wydzielić zespół działań, które składają się na pojedynczą iterację. Taką pojedynczą iterację można wykonać dowolnie dużo razy. Kolejne iteracje, mogą jedynie poprawić już znaleziony wynik.
   1.4. **Optymalizator bezparametrowy.** Parametry optymalizatora mają być niewidoczne dla użytkownika. Ich ustalenie może odbywać się w konstruktorze metody i być stałe, może zmieniać się w trakcie przebiegu itp. Ważne jest, że z punktu widzenia użytkownika optymalizator jest bezparametrowa.
   1.5. **Rozwiązywane problemy.** Rozwiązaniem problemu jest ciąg liczb naturalnych (całkowitych dodatnich). Długość ciągu dyskretnego oraz dziedzina każdej zmiennej są określane przez obiekt klasy definiującej problem. Podstawą konkursu będzie 240 instancji problemu LcVRP. Jakość rozwiązań będzie oceniana zgodnie z podaną definicją problemu.

2. Wymagania dla kodu.

2.1. **Język programowania.** Dostarczony kod musi być zgodny z wymaganiami laboratorium TEP. W szczególności niedozwolone jest używanie inteligentnych wskaźników w jakiejkolwiek formie (z wyłączeniem tych, które zostały napisane osobiście). Można używać *move semantics*.

2.2. **Zarządzanie pamięcią.** Wymagane jest ręczne zarządzanie pamięcią. W szczególności nie jest dopuszczalne używanie inteligentnych wskaźników (patrz punkt 2.1).

2.3. **Projekt.** Zgłaszając swoje rozwiązanie do konkursu należy wysłać spakowany projekt (wymagany format: **zip**). Ze spakowanego projektu należy usunąć foldery data, release, debug (również te wewnątrz projektu LcVRPContest) oraz plik *LcVRPContest.VC.db*. **Po spakowaniu i usunięciu zbędnych katalogów, rozmiar spakowanego projektu nie powinien przekraczać 50 kB. Paczki o rozmiarze powyżej 1MB nie będą przyjmowane.**

- Wymagane jest użycie projektu opublikowanego na ePortalu prowadzącego kurs. Jest to projekt w Visual Studio 2022. **Tylko takie projekty będą przyjmowane**.
- Spośród plików obecnych w projekcie **MODYFIKOWAĆ MOŻNA TYLKO PLIKI Optimizer.hpp i Optimizer.cpp**.
- Można dodawać do projektu nowe pliki, z których korzystać będą Optimizer.hpp i Optimizer.cpp
- **KONKURS DOTYCZY UMIEJĘTNOŚCI PROGRAMOWANIA. Ocena jakości zaproponowanych tras nie została zaimplementowana w sposób wydajny. Do kodu nie została dołączona także żadna dokumentacja. Zdolny programista może jednak z łatwością przeanalizować cudzy kod i napisać własny o równoważnej funkcjonalności, ale działający lepiej. W trakcie optymalizacji zdolny programista może używać własnej implementacji problemu (liczącej szybciej i dającej dostęp do dodatkowych statystyk/funkcjonalności). Jedynie ostateczne rozwiązanie zostanie ocenione przez oryginalną (niewydajną) klasę.**
- Warunkiem jest, że dla takiego pliku LcVRPContest.cpp jaki został podany w projekcie przykładowym, projekt będzie kompilował i wykonywał się prawidłowo. Rodzaj kompilacji: **x64 Relesase. Rada:** nie używaj „Start debugging". Używaj „Start **without** debugging"
- W klasie `Optimizer` niezmienne pozostają:
  - Nazwa klasy (`Optimizer`)
  - Konstruktor (`Optimizer(Evaluator& evaluator);`)
  - Metody:
    - `void Initialize();` (**uwaga**: należy modyfikować implementację metody)
    - `void RunIteration();` (**uwaga**: należy modyfikować implementację metody)
    - `vector<int>* GetCurrentBest() { return &current_best_; }` (**uwaga**: tutaj implementacja metody jest **niezmienna**)
  - Pola:
    - `Evaluator &evaluator_;`
    - `vector<int> current_best_;`

2.4. **Pozostałe wymagania.** Tworząc kod, należy kierować się wytycznymi wskazanymi na zajęciach TEP. Złamanie zasad może doprowadzić do dyskwalifikacji zaproponowanego rozwiązania, bez względu na jakość jego działania. Będzie to subiektywna decyzja organizatorów konkursu. Na przykład, jak można się domyślić, rzucanie wyjątków nie będzie mile widziane (uwaga: **eufemizm!**).
2.5. **Dla każdego przypadku testowego, metoda ma działać tak samo.** To znaczy, że nie można uzależniać działania metody od typu konkretnej załadowanej instancji problemu.
2.6. **Wątki. Można używać wielowątkowości.** Należy pamiętać, że równolegle (na komputerze o zadanej specyfikacji sprzętowej) zostanie uruchomionych wiele procesów **tego samego** zawodnika (patrz punkt IV). Procesy liczące dwóch różnych zawodników **nigdy** nie zostaną uruchomione równolegle.

3. Przypadki testowe.

3.1. **Liczba problemów.** Planujemy, że konkurs zostanie przeprowadzony dla 240 różnych przypadków testowych.

3.2. **Zadanie.** Zadaniem będzie odnalezienie rozwiązania o jak najmniejszej wartości (problem dotyczy minimalizacji).

3.3. **Liczba przebiegów i jakość rozwiązania.** Każdy optymalizator zostanie uruchomiony od 5 do 10 razy (zależy od liczby uczestników), raz dla każdej instancji testowej.

3.4. **Ranking.** Na podstawie wyników określonych zgodnie z miarami podanymi w poprzednim podpunkcie, zostanie utworzony ranking. Najlepszy optymalizator otrzyma dla danego przypadku testowego 1 miejsce, drugi 2 miejsce, itd. Jeśli więcej niż jeden optymalizator będzie miał to samo miejsce to wszystkie one otrzymają to samo, najwyższe możliwe miejsce. Na przykład:

Mamy 3 optymalizatory (A, B i C) i 2 instancje (instancja 1 i instancja 2). Jako podstawową wartość klasyfikacji bierzemy pod uwagę medianę, potem wartość najlepszą, potem maksymalną. Dla 2 problemów, wspomniane optymalizatory uzyskały następujące wyniki (im mniejsza wartość tym lepiej). Przykładowe wyniki są przedstawione w tabeli.

	Instancja 1			Instancja 2		
Optymalizator	**Med**	**Min**	**Max**	**Med**	**Min**	**Max**
Optymalizator A	0.8	0.8	0.8	0.6	0.3	0.6
Optymalizator B	0.8	0.8	1	0.5	0.4	0.7
Optymalizator C	0.8	0.8	0.8	0.6	0.3	0.6

Tabela 1. Przykładowe wyniki dla dwóch problemów i trzech metod.

Metoda	Instancja 1	Instancja 2
Optymalizator A	1	2
Optymalizator B	3	1
Optymalizator C	1	2

Tabela 2. Przykładowy ranking dla wyników wskazanych w **Tabela 1**.

Komentarz. Dla Instancji 1, optymalizatory A i C osiągają najlepsze wyniki. Optymalizator B ma tą samą medianę i tą samą wartość minimalną, ale wartość maksymalna jest gorsza niż dla metod A i C, dlatego metoda B zajmuje dla Instancji 1, 3 miejsce, a metody A i C 1 miejsce. Dla Instancji 2 sytuacja jest odmienna. Pierwsze miejsce zajmuje metoda B, bo jej wyniki mają najniższą medianę (to, że minimum i maksimum są niższe dla optymalizatorów A i C nie ma znaczenia). Optymalizatory A i C zajmują 2 miejsce. W ostatecznym rozrachunku, zwycięzcami są metody A i C (średnie miejsce: 1.5), a metoda B zostaje uznana za najgorszą (średnie miejsce: 2).

II. O problemie optymalizacji zestawu tras pojazdów dostawczych

Konkurs dotyczy problemu *Limited Capacitated Vehicle Routing Problem (LcVRP)*.

Dla zdefiniowania LcVRP najpierw zdefiniujemy problem ***Capacitated Vehicle Routing Problem (cVRP)*. Problem cVRP jest podobny do *Travelling Salesman Problem* (TSP).**

W TSP danych jest *n* klientów/miast i należy zaplanować trasę, która jest cyklem, czyli objeżdżamy wszystkich klientów/miasta w zadanej kolejności. W TSP minimalizujemy koszt tego przejazdu.

W cVRP jest tak samo, ale cykli może być *k*. Też jest *n* lokalizacji, ale klientów (czyli zmiennych, których dotyczy optymalizacja) jest w cVRP jest *n-1*, ponieważ pierwsza lokalizacja, z której startuje każdy cykl to magazyn (*depot*). Czyli, w cVRP przypisujemy każdego klienta do jednej z *k* grup i dla każdej z tych grup minimalizujemy koszt przejazdu. Na tej podstawie można powiedzieć, że TSP to szczególny przypadek cVRP, gdzie *k=1*.

Typowy sposób kodowania rozwiązania do cVRP jest następujący:

- Pierwsza część genotypu to permutacja (jak dla TSP, tak jakbyśmy mieli tylko jeden cykl), a druga to grupowanie klientów do *k* grup.
- Na przykład dla *n=6 (de facto n=5, bo manipulujemy tylko klientami, magazyn (depot) pozostaje niezmienny) i k=2* rozwiązanie cVRP może wyglądać tak:

	Część definiująca permutację					**Część definiująca grupowanie**				
Klient	**k1**	**k2**	**k3**	**k4**	**k5**	**k1**	**k2**	**k3**	**k4**	**k5**
genotyp	2	1	3	5	4	0	0	1	1	0

Takie rozwiązanie mówi nam, że:

- Pierwsza grupa (o numerze ***zero***, oczywiście, bo tak numerują programiści…) to klienci *k1, k2 i k5*
- Druga grupa (o numerze ***jeden***, też oczywiście…) to klienci *k3* i *k4*

Cykl dla pierwszej grupy to: **depot -> k2 -> k1 -> k5 -> depot**
Cykl dla drugiej grupy to: **depot -> k3 -> k4 -> depot**

Zauważ, że rozwiązanie, gdzie wszyscy klienci są ułożeni w *jeden* cykl (pierwsza część genotypu) po przefiltrowaniu grupowaniem (druga część genotypu) zawsze daje prawidłowe *k* cykli. Potencjalnie, druga część może wygenerować mniej niż *k* cykli.

Limited cVRP (LcVRP). Konkurs będzie dotyczyć problemu LcVRP. Instancja LcVRP to instancja cVRP, ale z zadanym cyklem. Tak więc w LcVRP optymalizacja dotyczy wyłącznie grupowania (druga część genotypu).

Przykładowe instancje znajdują się w paczce konkursowej w folderze data. Więcej informacji na temat problemu cVRP można znaleźć tutaj:
http://dimacs.rutgers.edu/programs/challenge/vrp/cvrp/
https://vrp.galgos.inf.puc-rio.br/index.php/en/

III. Uczestnicy

W konkursie może wziąć udział każdy chętny, który zgłosi prawidłowo swój program. Nie trzeba nawet być studentem. Oczywiście nagroda numer 5, będzie przyznawana wyłącznie studentom, którzy obecnie uczęszczają na laboratorium z TEP.

IV. Techniczne przeprowadzenie konkursu i wyniki

1. W konkursie wezmą udział te osoby, które zgłoszą swoje programy mailem do dnia 2026.01.25 (niedziela) do godziny 12:00. Jeżeli ktoś zgłosi swój program później to nie zostanie on wzięty pod uwagę. Paczka zip ma być wysłana na adres michal.przewozniczek@pwr.edu.pl. Paczka ma być stworzona zgodnie z opisem zamieszczonym w punkcie 2.3. **NIE WOLNO wysyłać paczek o rozmiarze powyżej 1MB**.

2. Jedna osoba może zgłosić tylko jeden program. W przypadku wysłania programu kilka razy zostanie wzięta pod uwagę ostatnia prawidłowo przesłana wersja.

3. Obliczenia zostaną przeprowadzone na serwerze: PowerEdge Dell server 2 * AMD EPYC 9654 (192 rdzenie, 384 procesory logiczne) 1.5TB RAM. W momencie wykonywania obliczeń na serwerze nie będą uruchomione inne procesy w znaczący sposób obciążające zasoby. Liczba uruchomionych jednocześnie programów nie będzie większa niż 60. W tym samym momencie będą uruchomione wyłącznie programy jednego zawodnika. Na tej podstawie można oszacować, ile wątków uruchomić, jeśli chcemy napisać program wielowątkowo. Można też wykorzystać inne mechanizmy, żeby dynamicznie dopasowywać liczbę wątków do aktualnego obciążenia procesora. Można też napisać program jednowątkowo.

4. Czas obliczeń wyniesie 3 godziny dla pojedynczego eksperymentu.

5. W trakcie wykonania programu, po każdej iteracji zapamiętywany będzie najlepszy odnaleziony dotychczas wynik. Jeśli ostatnia iteracja przekroczy dozwolony czas obliczeń, to jej wynik nie będzie brany pod uwagę. Na przykład:
   Dopuszczalny czas to 3 godziny. Po 2 godzinach i 58 minutach najlepszy wynik to 0.98, po 2 godzinach i 59 minutach 0.99, a po 3 godzinach i 2 minutach 1.00. W takiej sytuacji pod uwagę będzie brany wynik 0.99, ponieważ ostatni wynik został zwrócony po czasie.

6. Komplet wyników wraz z odpowiednim raportem zostanie ogłoszony na ostatnim wykładzie w semestrze (2026.01.30, piątek). Odpowiednie dane będą również dostępne na ePortalu.

7. **Udział w konkursie oznacza zgodę na publikację swoich danych osobowych wraz z wynikami konkursu.**

V. Nagrody

Główną nagrodą, dla każdego uczestnika konkursu jest **dobra zabawa** oraz **możliwość rozwiązania ciekawych i trudnych problemów za pomocą własnego optymalizatora**. Ponadto, do zdobycia są następujące nagrody:

1. **Niepowtarzalny, oczywiście limitowany, okolicznościowy dyplom** z oryginalnymi, odręcznymi podpisami organizatorów Konkursu.

2. Pięciosekundowa pochwała wzrokowa udzielona na ostatnim wykładzie (warunkiem udzielenia tej nagrody jest obecność na tym wykładzie, prowadzący kategorycznie odmawia spoglądania w próżnię).

3. Dla chętnych, którzy będą zainteresowani wdrażaniem nowoczesnych technik optymalizacji w problemach praktycznych proponujemy od 1 do 3 stypendiów w wysokości 5 kPLN netto przez okres 3-miesięcy (w sumie 15kPLN netto). Aby uzyskać stypendium należy wziąć udział w 5 spotkaniach Koła Naukowego *Genetic Algorithms 'n' Great Structure Analyzing Solvers* (GANGStAS), które odbędą się w semestrze letnim. Następnie zaproponować badania dla wybranego problemu praktycznego (prowadzący może w tym pomóc). Propozycje będą oceniane i możliwe jest ich odrzucenie. W przypadku przyjęcia większej niż 3 liczby aplikacji, stypendium otrzymają te 3 osoby, które zajęły wyższe miejsce w konkursie. W przypadku szczególnie ambitnych i obiecujących projektów możliwe jest wydłużenie stypendium na okres od 1 do 2 lat.

4. Możliwa dalsza współpraca, wyjazdy do wiodących ośrodków badawczych w Holandii (CWI Amsterdam, Utrecht University), Hiszpanii (University of Malaga) i innych. W ramach dalszej współpracy możliwe jest również uzyskanie stypendium w grancie prowadzącego, pod warunkiem dostępności środków.

5. **Last but not least**. Zdobywcy pierwszych dziesięciu miejsc mogą liczyć na **podniesienie oceny z laboratorium z TEP i wykładu z TEP o 1.0**. Druga dziesiątka może liczyć na podniesienie oceny z wykładu i laboratorium o 0.5.

VI. Komentarze i wskazówki

Poprawić jakość działania (mierzoną jakością rozwiązania) optymalizatora ewolucyjnego można na bardzo wiele sposobów. Można skoncentrować się na takiej optymalizacji kodu, która spowoduje, że optymalizator będzie działać szybciej. Na przykład, w kolejnych populacjach, niektóre osobniki (rozwiązania) są kopiami osobników (rozwiązań) już wcześniej ocenionych. Skoro tak, to nie trzeba oceniać ich powtórnie. Pominięcie niepotrzebnego oceniania osobnika może *znacząco* przyspieszyć metodę (*znacząco*, czyli na przykład 1000-krotnie).

Jeśli optymalizator utknie (co objawia się między innymi tym, że przez wiele iteracji nie poprawi najlepszego znalezionego wyniku) to można spróbować wykonać dodatkowe operacje, które mają temu zaradzić. Np. chwilowo zwiększyć rozmiar populacji, zmienić działanie optymalizatora (np. zwiększyć prawdopodobieństwo mutacji i/lub krzyżowania), albo zmienić sposób wyboru rodziców.

Można też wykonać hybrydę Algorytmu Genetycznego z jakimś algorytmem optymalizacji lokalnej i na tej podstawie generować nowe rozwiązania, które będą jakościowo lepsze niż rozwiązania generowane w pełni losowo.

Wreszcie, można zastanowić się czy między genami nie występują jakieś zależności. Jeśli tak, to można spróbować je wykryć i użyć tej wiedzy, żeby poprawić jakość optymalizacji.

Powyższe to tylko pomysły, które wpadły do głowy autorowi dokumentu. Mogą być dobre albo złe, ale liczba możliwości jest nieograniczona…

UDANEJ ZABAWY!!!